

DISCOVERY PHASE – COMPLETE (Zero assumptions; requirements locked verbatim from your query + full Sovereign Machina architecture + changelog v3.2.7 + latest repo state)

Core functionality required:

- Assess the difficulty of rebuilding **Droid CLI** as a **native Rust** implementation (from scratch or heavy port), using the claw-code Rust port as a reference.
- Evaluate feasibility, effort, benefits, and integration with the current Sovereign Machina tri/quad-node stack (Hermes supervisor, GBrain rewrite, Omi voice, etc.).
- Focus on standalone personal intelligence use while preserving sovereignty, zero-trust, and local-first design.
- Maintain: Node A and Node B completely unchanged in core logic.

All requirements are now exhaustive and production-locked.

RESEARCH PHASE – RIGOROUS CROSS-CHECK (claw-code repo + Droid CLI characteristics)

The claw-code repo is a **full, production-grade Rust rewrite** of a Claude Code-like CLI agent harness. It includes a complete Rust workspace (`rust/` directory), cargo build system, session management, tool calling, persistent state, and a clean CLI surface. It is actively maintained, has parity tracking, and demonstrates that porting a complex agent CLI to Rust is not only possible but results in a high-performance, native binary.

Droid CLI (FactoryAI) is a Rust-based binary already (from public info), but the user wants a **sovereign-native rebuild** optimized for Hermes integration, persistent Droid-like sessions, and your stack (MCP/VSB, MemPalace, GBrain, etc.).

DIFFICULTY ASSESSMENT & RECOMMENDATION

Overall Difficulty: Medium to High (3–8 weeks for a solid MVP, depending on scope).

Why Feasible:

- claw-code proves that a full agent CLI (tool calling, sessions, persistent state, orchestration) can be built cleanly in Rust.
- Droid CLI is already Rust-native in parts, so you are not starting from zero — you can mine patterns for filesystem isolation, credential handling, resumable jobs, and remote orchestration.

- Your existing Rust sidecars (artery daemon, Sovereign Observer) and Hermes provide a strong foundation for integration.

Key Challenges:

- **Stateful Execution:** Replicating Droid’s persistent filesystem, credential store, and resumable sessions requires careful design (sandboxing, versioned state, crash recovery).
- **Tool Integration:** Droid has rich tool calling and parallel execution — you need to map this to your MCP/VSB and Hermes enrichment.
- **Security:** Credential handling and sandboxing must be stricter than the original to maintain sovereignty.
- **Scope Creep:** Avoid copying every Droid feature — focus on what strengthens Hermes (persistent background agents, session continuity, remote queuing from Machina Terminal).

Recommended Path: Build a Sovereign Droid Runtime as a Rust crate on top of Hermes
(not a full from-scratch Droid CLI clone).

- Start with a minimal viable runtime (persistent sessions, job queue, filesystem isolation).
- Integrate it as a service on the Hermes Pod.
- Use claw-code as a reference for Rust patterns (session store, tool harness, CLI surface).

High-Level Rust Crate Structure (sovereign-droid-runtime)

Rust

```

sovereign-droid-runtime/
├─ src/
│   ├─ lib.rs                # Main runtime entrypoint
│   ├─ config.rs             # Config (paths, sandbox policy, MCP)
│   ├─ session/
│   │   ├─ mod.rs
│   │   └─ store.rs          # Persistent session state (SQLite +
Mooncake spillover)
│   │       └─ manager.rs     # Session lifecycle
│   └─ fs/
│       ├─ mod.rs
│       └─ isolated.rs       # Per-session sandboxed filesystem
├─ queue/
│   └─ mod.rs
│       └─ minion.rs         # Durable, resumable job queue
├─ vault/
│   └─ mod.rs
│       └─ secure.rs         # Credential proxy (steganography +
AES)
├─ hermes_bridge.rs          # Integration with existing Hermes
supervisor
├─ executor.rs              # Safe task execution with sandboxing
└─ Cargo.toml

```

Integration Benefits

- Hermes enriches voice intents and queues jobs to the Droid runtime.
- Node B (Director) gets clean, structured results.
- Full session continuity and background execution without relying on external Droid CLI.
- Easier to make sovereign (no Factory cloud coordination).

Effort Estimate

- MVP (basic sessions + job queue): 2–4 weeks.
- Full feature parity with Droid Computers: 6–10 weeks.
- Leverage claw-code patterns to accelerate.

ARCHITECT SIGN-OFF

Rebuilding Droid CLI as a native Rust runtime on top of Hermes is feasible and a strong long-term move. The claw-code repo provides an excellent reference for Rust agent CLI patterns (session management, tool harness, persistent state). A Sovereign Droid Runtime crate layered on Hermes gives you persistent, resumable agents, session continuity, and background execution while staying fully under your control. Start with a minimal viable version on the Hermes Pod — it will deepen your self-improving assistant without disrupting the current stack. Rust is the right choice for safety and performance in a stateful runtime.

Your main sovereign machine remains completely untouched and rock-solid.

Clarifying questions (mandatory before next phase):

1. Do you want the starter Rust crate structure (`sovereign-droid-runtime`) with session store and minion queue?
2. Shall I include integration points with Hermes and Machina Terminal for voice-triggered background tasks?

Reply with your priority and I will deliver the concrete starter code. The native Droid runtime is ready to be prototyped.